Reflection Budgets in Agentic Context Engineering

FTEC5660 reproducibility study
Formula benchmark · DeepSeek Flash · November 2026

Abstract

This study runs the official ACE implementation on a reduced Formula benchmark and changes one algorithmic parameter: the maximum number of correction-reflection rounds, from three to one. Across three sampling seeds, each with 50 training, 30 validation and 100 test questions, the empty-playbook control achieves 70.3%, three-round ACE 78.7%, and one-round ACE 79.0% mean exact-match accuracy. The three-round change against the control is +8.3 percentage points; changing three rounds to one changes mean accuracy by +0.3 points. These are measurements under a substituted model and reduced data, not an exact reproduction of the paper's absolute scores.

Questions and contribution

RQ1: Does the experience accumulated by ACE improve held-out financial numerical reasoning in this resource-constrained setting? RQ2: How does a one-round correction limit affect accuracy, realized reflection work and cost compared with three rounds? The contribution is a documented reproduction and a controlled parameter study, not a new algorithm.

Configuration	Mean accuracy	Sample SD	Runs
Empty playbook	70.3%	3.21 pp	3
ACE: max 3 rounds	78.7%	2.89 pp	3
ACE: max 1 round	79.0%	5.00 pp	3

All runs are retained. A negative or null effect is reported as such. The three seeds vary data subsets and order; provider stochasticity is not independently controlled. Overlapping test subsets across seeds preclude treating 300 predictions per arm as 300 independent questions.

1. Reproduction target and system architecture

The target is the direction of the supervised offline Formula result in ACE Table 2: the paper reports 67.5% for its DeepSeek-V3.1 base model and 85.5% for ACE. This study evaluates a smaller subset with a current hosted model. The paper's reflection-depth sensitivity experiment is on AppWorld (Appendix A.6, Table 19); our one-versus-three comparison transfers that question to Formula. [1]

Component	Role in the executed pipeline
Generator	Answers a question using the current playbook and optional reflection.
Evaluator	Compares the extracted answer with the training label or held-out label.
Reflector	Analyzes the answer, using training ground truth during adaptation.
Curator	Proposes structured incremental updates to the playbook.
Playbook	Persists reusable rules across training examples; frozen for held-out tests.

A training step performs initial generation, conditional correction rounds, curation, and a post-curation generation. A correct initial answer still triggers a reflection to tag useful entries. Incorrect answers stop correction early when they become correct. Consequently, a maximum of three rounds does not imply three rounds on every example. [2]

The small modification changes only max_num_rounds. The base arm uses the same Generator with an empty playbook. The underlying LLM weights are not trained. This is context-level self-improvement; the experiment does not establish recursively improving the improvement algorithm itself.

Code map: ace/ace.py orchestrates learning; ace/core/{generator,reflector,curator}.py implement roles; eval/finance/data_processor.py implements the task metric. The local scripts/run_experiment.py supplies a compatible client and records usage without editing upstream algorithm files. [2]

2. Experimental protocol and reproducibility

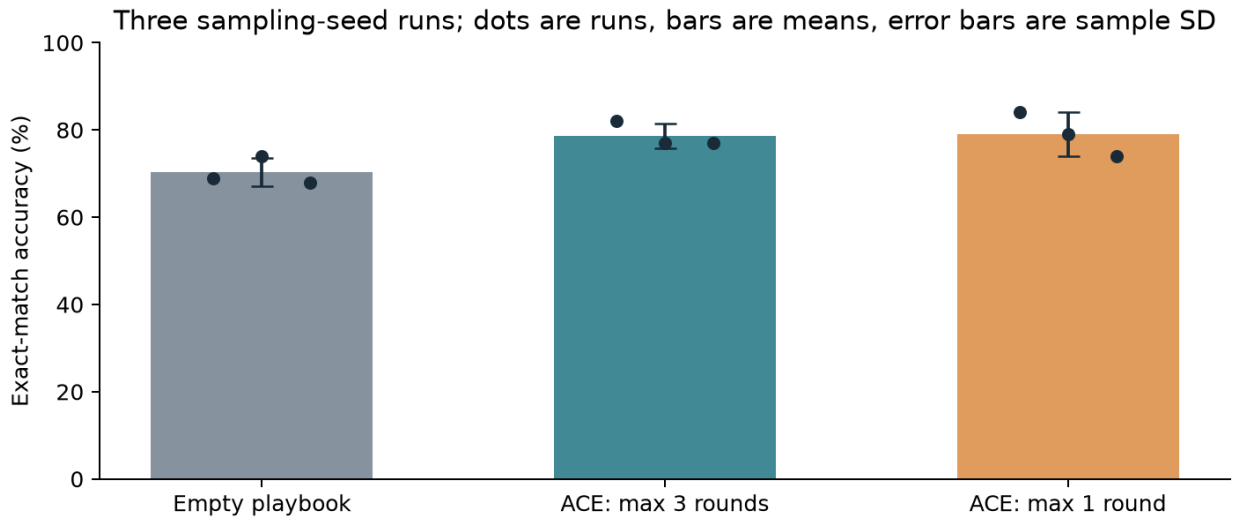
Setting	Executed value
Upstream	82709de050e1db6e6ef2f07bcb0393560b94992a
Model / provider	deepseek-flash / api.deepseek.com
Generation	Thinking disabled; temperature 0; max output 2048
Training	One epoch; 50 examples; empty initial playbook
Validation / test	30 / 100 examples per seed
Seeds	42, 43, 44; Python sampling, not provider RNG
Evaluation	Four workers; exact-match Formula processor
Memory controls	80,000-token playbook budget; embedding dedup disabled
Resource	Windows CPU orchestration; hosted API; no local LLM training

The repository contains 500 train, 300 validation and 200 test records. Exact duplicate contexts were removed across split boundaries before deterministic sampling; none were found. Each seed uses identical subsets across all three arms. Source and subset hashes accompany the deliverable. Semantic near-duplicates and possible pretraining exposure were not ruled out.

Only an end-of-epoch validation check is made. The selected playbook is therefore effectively the final trained checkpoint, except for the upstream empty-playbook fallback if validation accuracy is zero. No hyperparameters are selected from the final test results. The pilot was used for execution debugging, not prompt or algorithm tuning; pilot/main question overlap is disclosed.

Important comparability limits: the model, dataset sizes, output cap, optional deduplication and adaptation schedule differ from the original study. Our control uses ACE's Generator prompt rather than the paper's dataset-default prompt. All test denominators are audited as exactly 100.

3. Accuracy results



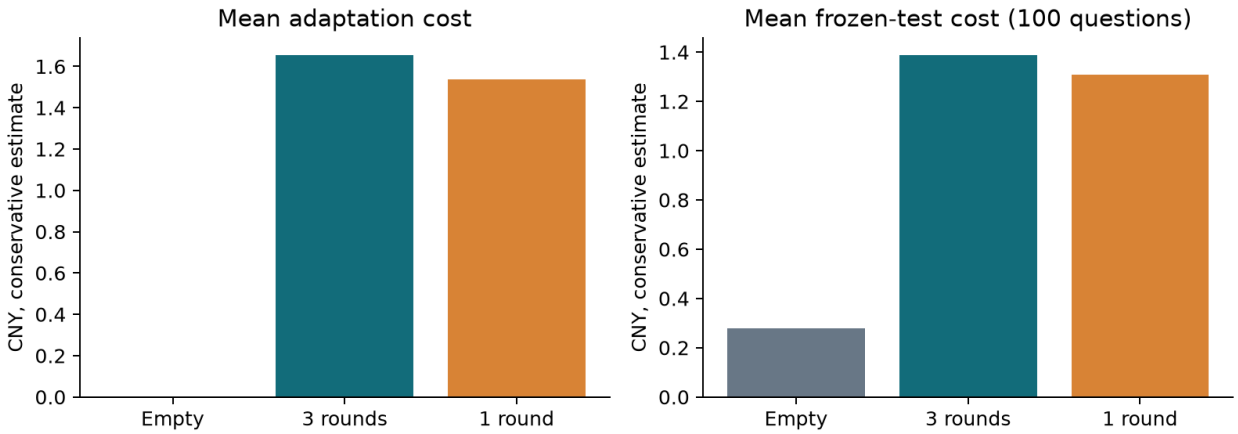
Seed	Empty	ACE 3: initial / final	ACE 1: initial / final
42	69.0%	71.0% / 82.0%	69.0% / 84.0%
43	74.0%	71.0% / 77.0%	72.0% / 79.0%
44	68.0%	70.0% / 77.0%	68.0% / 74.0%

The mean three-round-minus-control difference is +8.3 pp; one-round-minus-three-round is +0.3 pp. The paper's 85.5% reference is not a directly comparable pass/fail threshold. Repeating initial empty-playbook evaluation inside the ACE arms makes model variability visible rather than treating a single baseline realization as deterministic.

Arm	Sample variance (accuracy fraction²)	Mean ± sample SD
Empty playbook	0.001033	70.3 ± 3.21 pp
ACE: max 3 rounds	0.000833	78.7 ± 2.89 pp
ACE: max 1 round	0.002500	79.0 ± 5.00 pp

Mean, SD and variance summarize three runs. These are descriptive statistics, not a confidence interval for general performance. Per-run Wilson intervals and exploratory paired discordance tests are in analysis.json. Overlap across seeds and the small number of runs limit inferential claims.

4. Cost and realized reflection work

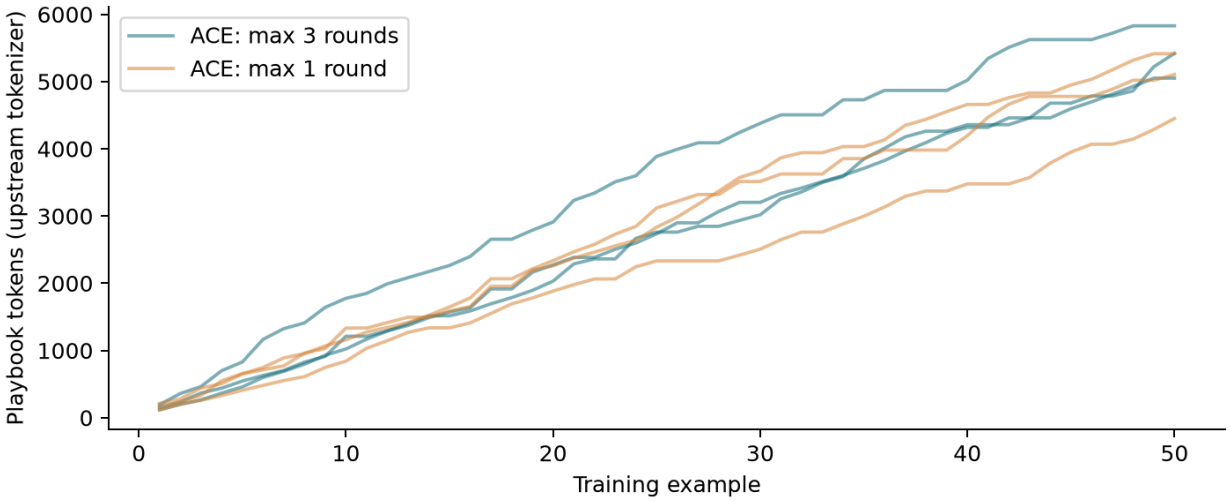


Arm / seed	All calls	Correction rounds	Beyond first	All-run cost
base / 42	100	0	0	0.291
ace_r3 / 42	443	11	2	3.976
ace_r1 / 42	438	8	0	3.559
base / 43	100	0	0	0.263
ace_r3 / 43	441	10	1	3.556
ace_r1 / 43	440	10	0	3.718
base / 44	100	0	0	0.285
ace_r3 / 44	445	13	2	3.619
ace_r1 / 44	440	10	0	3.218

Costs use returned prompt/completion tokens at the verified peak cache-miss rates of CNY 2 / 8 per million, respectively. Cache hits and off-peak pricing can make the actual invoice lower. No claim is made to have reconciled the account invoice. [3]

All-run cost includes initial tests and validation in addition to adaptation and final tests. The separate final-test figure excludes learning. Correction rounds count Reflector calls on wrong initial answers; correct-answer reflection remains present in both ACE arms. Differences in evolving trajectories can outweigh the nominal round cap.

5. Experience growth and paired error analysis



Seed	3-round → 1-round gains	Losses	Net pp
42	2	0	2
43	5	3	2
44	0	3	-3

Gains are questions the three-round run misses and the one-round run answers correctly; losses reverse that relationship. These matched-question counts distinguish a small net change from extensive answer churn. They do not identify which learned rule caused a particular outcome.

Deterministically selected error example (seed 42, ace_r3, lowest wrong index 1): predicted 37161.61; target 37161.62. This example is chosen by index rather than rhetorical appeal. The full question and output are preserved in the local data and raw logs.

Deterministically selected error example (seed 42, ace_r1, lowest wrong index 1): predicted 37161.6; target 37161.62. This example is chosen by index rather than rhetorical appeal. The full question and output are preserved in the local data and raw logs.

Playbook token growth is measured by the upstream cl100k_base counter, not the provider billing tokenizer. Larger memory is not in itself evidence of useful learning. Exact-match failures include arithmetic, sign, units or answer formatting; this report does not silently introduce a tolerance to improve the score.

6. Debug diary and validity checks

Issue	Resolution / consequence
GitHub clone and API access	Clone stalled; directory API was rate-limited. Used official codeload archive and verified its commit marker.
Model-provider compatibility	Injected an OpenAI-compatible DeepSeek client; preserved original role prompts and learning logic.
Unbounded retries	SDK retries disabled. Budget/transport failures stop execution, including through upstream exception handlers.
Budget persistence	Reserve before requests; settle known usage at conservative rates; retain unknown-request reserves; prevent concurrent runner processes.
Optional embedding dependencies	Disabled consistently; used CPU orchestration with minimal dependencies and a pinned environment.
Scoring denominator	Require all 100 questions and consistency between correct counts and wrong-index sets.
Randomness	Temperature zero did not make repeated outputs identical. Report all three seeds and initial-test repeats.

Integrity tests cover budget persistence, settlement and refusal above the limit. Python compilation and end-to-end API runs provide separate checks. An immutable source commit, dependency lock, data hashes, raw responses and playbook snapshots support independent inspection.

Limitations: only one task and one current hosted-model alias; three overlapping sampling seeds; no separation of sampling variance from API randomness; fixed execution order; possible response truncation under the 2048-token cap; supervised training feedback; no causal attribution of individual rules; no reproduction of AppWorld tool-use behavior. A small subset may not expose benefits requiring longer adaptation.

7. Conclusions and reproducibility package

The implementation reproduces the operational behavior of experience accumulation and frozen-playbook evaluation. Under the stated conditions, three-round ACE changes mean accuracy by +8.3 pp relative to the empty-playbook control. Reducing the cap to one changes mean accuracy by +0.3 pp relative to three rounds. These results answer the selected task under a constrained configuration; they do not establish the original paper's absolute performance or broad superiority of either cap.

The most useful follow-up is to replicate with an identical held-out set and repeated provider calls, then vary training volume while holding model and prompts fixed. Any future prompt or evaluator change should be a separate experiment. More reflection should be justified by measured correction gains and final-task performance, rather than assumed to help.

What another student can reproduce

Fetch the pinned upstream archive, install requirements-lock.txt, run prepare_data.py, configure DEEPSEEK_API_KEY locally, and execute run_suite.py. Analyze the nine completed runs with analyze.py. Rebuild this report with build_report.py. API output may vary; the released measurements remain auditable through the accompanying evidence files.

References

[1] Zhang et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. arXiv:2510.04618v3, 2026. Table 2; Appendix A.6. https://arxiv.org/html/2510.04618v3

[2] ACE official implementation, commit 82709de050e1db6e6ef2f07bcb0393560b94992a. https://github.com/ace-agent/ace

[3] DeepSeek official model/pricing documentation, checked 8 November 2026. https://api-docs.deepseek.com/zh-cn/quick_start/pricing/

Use of AI assistance

AI assistance was used for code preparation, experiment orchestration, analysis and document drafting. All numerical results in this report come from saved API executions and deterministic analysis scripts. The submitting student should review the code, verify conclusions and follow course disclosure requirements.